

1) Importación de datos y exploración previa

```
In [8]: import pandas as pd
```

```
df = pd.read_table('C:/data/reviews_filmaffinity.csv', sep='\\|\\|', header=0)
df.head()
```

```
C:\Users\56951\AppData\Local\Temp\ipykernel_18444\3490258094.py:3: ParserWarning:
Falling back to the 'python' engine because the 'c' engine does not support regex
separators (separators > 1 char and different from '\s+' are interpreted as rege
x); you can avoid this warning by specifying engine='python'.
```

```
df = pd.read_table('C:/data/reviews_filmaffinity.csv', sep='\\|\\|', header=0)
```

Out[8]:

	film_name	gender	film_avg_rate	review_rate	review_title	review_text
0	Ocho apellidos vascos	Comedia	6,0	3.0	OCHO APELLIDOS VASCOS...Y NINGÚN NOMBRE PROPIO	La mayor virtud de esta película es su existen...
1	Ocho apellidos vascos	Comedia	6,0	2.0	El perro verde	No soy un experto cinéfilo, pero pocas veces m...
2	Ocho apellidos vascos	Comedia	6,0	2.0	Si no eres de comer mierda... no te comas esta...	Si no eres un incondicional del humor estilo T...
3	Ocho apellidos vascos	Comedia	6,0	2.0	Aida: The movie	No sé qué está pasando, si la gente se deja ll...
4	Ocho apellidos vascos	Comedia	6,0	2.0	UN HOMBRE SOLO (Julio Iglesias 1987)	"Pero cuando amanece,y me quedo solo,siento en...

El data set tiene 8602 registros y 8 columnas; en el campo "review_text" constan los comentarios respecto a cada filme, "film_name" indica el nombre de la película. La columna "review_rate" contiene la calificación asignada, la cual se categoriza de la siguiente manera: "positivo" si la nota es mayor a 6, "negativo" si es menor a 4, o "neutro" en caso contrario.

In [9]: # 1.- Nueva columna con la polaridad de los votos

[illegible]

Out[9]:	film_name	gender	film_avg_rate	review_rate	review_title	review_text	polaridad
0	Ocho apellidos vascos	Comedia	6,0	3.0	OCHO APELLIDOS VASCOS...Y NINGÚN NOMBRE PROPIO	La mayor virtud de esta película es su existen...	negativo
1	Ocho apellidos vascos	Comedia	6,0	2.0	El perro verde	No soy un experto cinéfilo, pero pocas veces m...	negativo
2	Ocho apellidos vascos	Comedia	6,0	2.0	Si no eres de comer mierda... no te comas esta...	Si no eres un incondicional del humor estilo T...	negativo
3	Ocho apellidos vascos	Comedia	6,0	2.0	Aida: The movie	No sé qué está pasando, si la gente se deja ll...	negativo
4	Ocho apellidos vascos	Comedia	6,0	2.0	UN HOMBRE SOLO (Julio Iglesias 1987)	"Pero cuando amanece,y me quedo solo,siento en...	negativo

```
In [40]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8603 entries, 0 to 8602
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   film_name              8603 non-null   object
1   gender                 8603 non-null   object
2   film_avg_rate          8603 non-null   object
3   review_rate            8601 non-null   float64
4   review_title           8603 non-null   object
5   review_text            8603 non-null   object
6   polaridad              8603 non-null   object
7   texto                  8603 non-null   object
dtypes: float64(1), object(7)
memory usage: 537.8+ KB
```

```
In [10]: #Concatenación de título y crítica

df['texto'] = df['review_title'] + ' ' + df['review_text']
```

```
In [49]: # Pasamos a array de numpy el texto (X) y el target-polaridad (y)

X = df['texto'].values
y = df['polaridad'].values

X_positivo = df[df['polaridad']=='positivo'].values
X_negativo = df[df['polaridad']=='negativo'].values
```

2.Preprocesamiento de texto

2.1 Importación de librería

```
In [12]: import nltk
nltk.download('stopwords')
nltk.download('wordnet')
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
from nltk.stem.snowball import SnowballStemmer
from nltk.stem.wordnet import WordNetLemmatizer
import pandas as pd
import re
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\56951\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]   C:\Users\56951\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

2.2 Limpieza, tokenización y reducción del vocabulario

```
In [56]: from nltk.tokenize import RegexpTokenizer

def clean(corpus):
    nltk.download('stopwords')
    stop_words = set(stopwords.words('spanish'))
    # en esta parte se tokeniza por expresión regular
    tokenizer = RegexpTokenizer('\w+|[\d\.]+|S+')
    cleaned_corpus = pd.Series()
    for doc in corpus:

        tokens= tokenizer.tokenize(doc)
        qs=[]
        for token in tokens:
            token=token.lower()
            if (not re.match("(|)|(|)|(|)", token)) and (token not in stop_words)
                qs.append(token)
        cleaned_corpus=cleaned_corpus.append(pd.Series(' '.join(qs)))
    return cleaned_corpus
```

```
In [25]: import spacy

def lematize(corpus):
    nlp = spacy.load('es_core_news_sm')
    lema_corpus = pd.Series()

    for doc in corpus:
        lemas = nlp(doc)
        qs=[]
        for lema in lemas:
            if not lema.is_punct:
                qs.append(lema.orth_)
        lema_corpus=lema_corpus.append(pd.Series(' '.join(qs)))
    return lema_corpus
```

```
In [26]: X_limpio = clean(X)
```

```
[nltk_data] Downloading package stopwords to  
[nltk_data] C:\Users\56951\AppData\Roaming\nltk_data...  
[nltk_data] Package stopwords is already up-to-date!
```

```
In [27]: X_preprocesado = lematize(X_limpio)
```

```
In [28]: X_preprocesado.head()
```

```
Out[28]: 0    ocho apellidos vascos ningún nombre propio may...  
0    perro verde experto cinéfilo pocas veces tan j...  
0    si comer mierda comas mierda si incondicional ...  
0    aida the movie sé pasando si gente deja llevar...  
0    hombre solo julio iglesias 1987 pero amanece q...  
dtype: object
```

2.3 Split de la muestra

```
In [29]: from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_preprocesado, y, test_size=0.2)  
  
print('Textos de entrenamiento: {}'.format(len(X_train)))  
print('Textos de test: {}'.format(len(X_test)))
```

```
Textos de entrenamiento: 6882
```

```
Textos de test: 1721
```

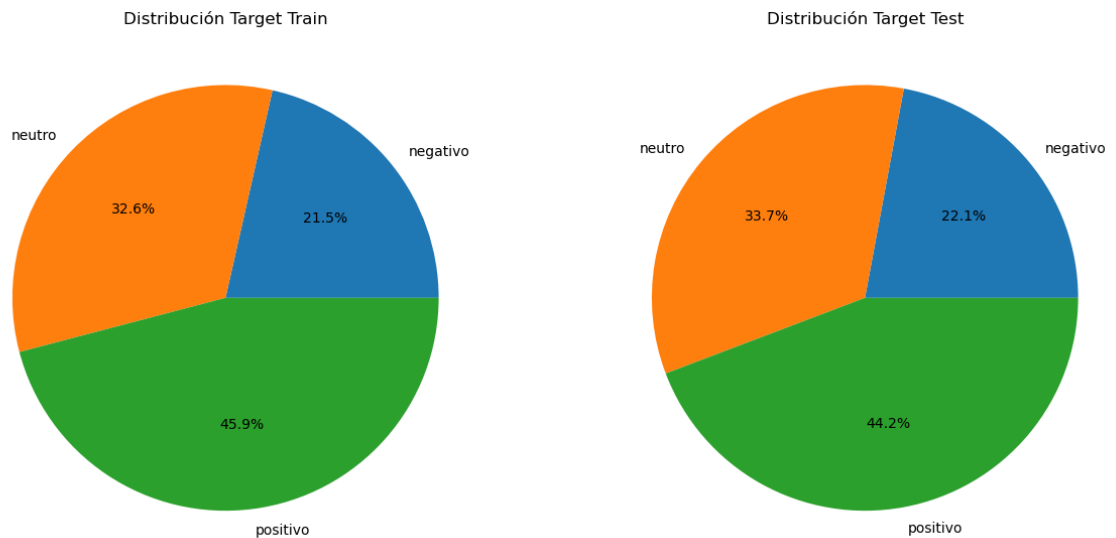
Este procedimiento asigna 6882 registros para el entrenamiento (80%) y 1721 para el test (20%). Para analizar la representatividad de estas muestras, realizamos una distribución de la polaridad (target en porcentaje) sobre cada split.

2.4 Distribución del target según muestra train y test

```
In [31]: import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# contamos las categorías del set train y guardamos sus etiquetas  
keys_train, counts_train = np.unique(y_train, return_counts=True)  
# contamos las categorías del set test y guardamos su etiquetas  
keys_test, counts_test = np.unique(y_test, return_counts=True)  
# calculamos los porcentajes del set train  
perct_train = counts_train / np.sum(counts_train)  
# calculamos los porcentajes del set test  
perct_test = counts_test / np.sum(counts_test)  
  
# definimos el tamaño del gráfico  
plt.figure(figsize=(15, 15))  
# grid primer gráfico: 1era fila, 2da columna, 1er gráfico  
plt.subplot(1, 2, 1)  
# se dibuja el primer pie  
plt.pie(perct_train, labels=keys_train, autopct='%1.1f%%')  
plt.title('Distribución Target Train')  
# grid primer gráfico: 1era fila, 2da columna, 2do gráfico  
plt.subplot(1, 2, 2)  
# se dibuja el segundo pie
```

```
plt.pie(perct_test, labels=keys_test, autopct='%1.1f%%')
plt.title('Distribución Target Test')
plt.plot()
```

Out[31]: []



Observamos que la distribución de la polaridad entre la muestra de desarrollo y validación es prácticamente la misma; por lo tanto, vemos que el split realizado es aceptable e insesgado. Por otro lado, este gráfico ilustra que no es necesario un balance de la muestra, debido que ninguna categoría es demasiado pequeña en porcentaje.

3. Representación por medio de vectores BoW

```
In [32]: from sklearn.feature_extraction.text import CountVectorizer

bow = CountVectorizer(max_features=2000, min_df=3)

# Creamos el modelo de bolsa de palabras con los textos de entrenamiento y aplicamos
X_bow_train = bow.fit_transform(X_train)

# A modo de ejemplo mostramos las 20 primeras palabras de la bolsa de palabras
bow.get_feature_names()[0:21]
```

```
Out[32]: ['10',
          '100',
          '13',
          '15',
          '20',
          '2004',
          '2014',
          '211',
          '30',
          '3d',
          '50',
          '51',
          '80',
          '90',
          'abajo',
          'abre',
          'absoluta',
          'absolutamente',
          'absoluto',
          'absurda',
          'absurdo']
```

```
In [61]: print(X_bow_train.toarray().shape)

(6882, 2000)
```

```
In [33]: X_bow_test = bow.transform(X_test)
```

3. Modelamiento Machine Learning

```
In [35]: from sklearn.naive_bayes import GaussianNB
          from sklearn.naive_bayes import MultinomialNB

          model= GaussianNB()
          model.fit(X_bow_train.toarray(), y_train)
```

```
Out[35]: GaussianNB()
```

```
In [41]: from sklearn.naive_bayes import MultinomialNB

          model= MultinomialNB()
          model.fit(X_bow_train.toarray(), y_train)
```

```
Out[41]: MultinomialNB()
```

Evaluación del modelo

```
In [44]: from sklearn.metrics import accuracy_score, precision_score, recall_score

          y_pred_train = model.predict(X_bow_train.toarray())
          y_pred_test = model.predict(X_bow_test.toarray())

          print('Accuracy score (train): {0:0.4f}'.format(accuracy_score(y_train, y_pred_train)))
          print('Accuracy score (test): {0:0.4f}'.format(accuracy_score(y_test, y_pred_test)))
          print('Precision score (train): {0:0.4f}'.format(precision_score(y_train, y_pred_train)))
          print('Precision score (test): {0:0.4f}'.format(precision_score(y_test, y_pred_test)))
          print('Recall score (train): {0:0.4f}'.format(recall_score(y_train, y_pred_train)))
          print('Recall score (test): {0:0.4f}'.format(recall_score(y_test, y_pred_test, average='micro')))
```

Accuracy score (train): 0.7036
Accuracy score (test): 0.6363
Precision score (train): 0.7083
Precision score (test): 0.6392
Recall score (train): 0.7036
Recall score (test): 0.6363

Según estos resultados, y fijándonos sobre todo en los índices de la muestra test, podemos concluir que el modelo es medianamente aceptable, y la siguiente tarea sería tratar de mejorarlo; para ello podríamos intentar otros modelos tales como: regresión logística multinomial, un support vector machine, random forest entre otros, incluso algún tipo de red neuronal, pero esta tarea se la dejamos a los estudiantes, ya que el objetivo de este ejercicio es sólo mostrar el procesamiento de texto, dentro del flujo de una aplicación Machine Learning. Otro aspecto que podría explorarse, con el objetivo de mejorar el performance del modelo, es intentar utilizar otra representación de los vectores tales como TF-IDF o incluso un Doc2Vec; tareas que de seguro serán muy interesantes para los estudiantes.